

FIT5225 – Assignment 1: CloudEco

Generative AI Prompts and Outputs Log

Wildfire and Smoke Detection on Kubernetes

Surya Ganesh | Student ID: 35415940

Model 1 – Wildfire and Smoke Detection (Fire, Smoke)

Public Endpoint: <http://169.224.230.182:30080>

April 2026

Responsible GenAI Use Statement

Generative AI was used openly as an assistant for planning, debugging, code explanation, command generation, benchmark interpretation, documentation drafting, and interview preparation throughout this assignment. Every piece of AI output was reviewed, tested, and verified by me before it was used in the submitted work. I ran all commands myself, tested all endpoints, and confirmed all Kubernetes deployments on the actual Oracle Cloud cluster. The final submitted code, configuration files, and report represent my own implementation and understanding of the system.

AI tools can produce incorrect or outdated information, so I cross-checked all generated code against the FastAPI documentation, Ultralytics documentation, Kubernetes documentation, and the assignment specification. Where the AI output did not match the requirements I corrected it.

How This Log is Organised

Each entry below contains the prompt I sent to the AI tool, a summary of the AI output that was useful, any representative commands the AI generated, and the action I took after verifying the output. Where the same topic required multiple follow-up prompts, these have been consolidated into a single entry so the log is readable. The entries follow the order in which work was completed.

1. Requirements Breakdown and Project Plan

Prompt

I have a university assignment called CloudEco for FIT5225. The task is to build a machine learning inference web service using FastAPI, containerise it with Docker, deploy it on a Kubernetes cluster running on Oracle Cloud VMs, and then benchmark it with Locust. I am assigned Model 1 which does wildfire and smoke detection using YOLO. The assignment is worth 30 percent and there is a live interview at the end which is a hurdle. Can you break the whole thing down into a step-by-step implementation plan I can follow? Include everything I need to deliver: the API, Docker image, Kubernetes deployment, IaC scripts, load testing, benchmark report, and the submission ZIP.

AI Response

The AI broke the project into seven sequential stages. Stage one was to set up the FastAPI application with `/health`, `/api/predict` and `/api/annotate` endpoints using a Pydantic request model that accepts a uuid and a base64-encoded image string. It noted that `/api/predict` must return detections, bounding boxes and YOLO timing metrics in the exact schema specified in the assignment.

Stage two was to integrate the Ultralytics YOLO model. The AI explained that the model weights would be provided and that inference should be called on a decoded NumPy or OpenCV image. It warned that running synchronous inference inside an async route handler would block the FastAPI event loop, causing all other requests to queue behind it, and that this must be handled properly to avoid catastrophic latency.

Stage three was Dockerfile engineering. The AI recommended using `python:3.11-slim` as the base image, installing CPU-only PyTorch using the `torch index` flag to avoid pulling a multi-gigabyte CUDA build, copying `requirements.txt` before source code so Docker layer caching works correctly, and running the container as a non-root user for security compliance.

Stage four was pushing the image to Docker Hub so Kubernetes worker nodes can pull it, since a local image is not accessible from remote cluster nodes. Stage five was writing Kubernetes manifest files: `namespace.yaml`, `deployment.yaml` with 1 vCPU and memory limits enforced, readiness and liveness probes pointing at `/health`, and `service.yaml` exposing the application via NodePort 30080.

Stage six was writing the Locust load generation script to send concurrent base64 image requests and record RPS, average latency, and P95 latency for 1, 2, 4 and 6 pod configurations. Stage seven was the benchmark report (max 500 words) with a results table, two plots, and analysis using Little's Law and queuing theory.

The AI also flagged the interview hurdle clearly: even a perfect submission results in a maximum of 45 marks if the student cannot explain their own code, demonstrate the live cluster, and walk through the Locust results. It advised preparing explanations for `run_in_threadpool` concurrency, Dockerfile layer optimisation, `kubectl` scaling commands, and how the LoadBalancer distributes traffic.

Student Verification and Action

I used the plan as my implementation checklist, ticking each item off against the assignment PDF and marking rubric before moving to the next stage.

2. FastAPI Application Structure

Prompt

Create a clean FastAPI application structure for wildfire and smoke detection. I need /health, /api/predict and /api/annotate routes. Use a Pydantic request model with a uuid string and a base64 image string. The /api/predict route must return uuid, count, detections as a list of class names, boxes as a list of objects with x, y, width, height and probability, and the three YOLO speed fields: speed_preprocess_ms, speed_inference_ms and speed_postprocess_ms. Add comments to every function because I have to explain this during a live interview.

AI Response

The AI generated a two-file layout. app/main.py contained the FastAPI app instance, the Pydantic request model ImageRequest with uuid and image fields, and three route handlers. app/utils.py contained the helper functions for decoding, inference and encoding so the route file stayed clean and readable.

The /health route returned {"status": "healthy"} as a basic liveness check. The /api/predict route accepted an ImageRequest body, called a decode helper, passed the result to the YOLO inference function, extracted bounding boxes and class labels, and returned the full response dictionary. The /api/annotate route followed the same pattern but called a separate annotation helper that used the Ultralytics plot() utility to draw bounding boxes on the image and returned the result as a base64-encoded JPEG.

Each function included a docstring explaining the input and output, and inline comments on non-obvious lines such as the base64 decode step, the xyxy-to-xywh bounding box conversion, and the speed metric extraction from the Ultralytics result object. The AI noted that the Pydantic model would automatically validate that both fields were present and return a 422 Unprocessable Entity if either was missing, which counts as error handling for invalid payloads.

The app was structured to be run with uvicorn app.main:app --host 0.0.0.0 --port 8000 either locally or inside Docker.

Student Verification and Action

I copied the structure into the project, ran uvicorn locally, and confirmed /health returned HTTP 200 with the expected JSON body.

3. Base64 Decoding, YOLO Inference and Response Formatting

Prompt

Write the helper functions for app/utils.py. I need: a function that takes a base64 string and returns an OpenCV image array, a function that runs the YOLO model on the image and returns detections in the assignment response format, a function that runs annotated inference and returns a base64-encoded JPEG. Make sure bounding boxes are in x, y, width, height format not xyxy, and extract speed_preprocess_ms, speed_inference_ms and speed_postprocess_ms from the Ultralytics results object.

AI Response

The AI provided four functions. `decode_base64_image` took the base64 string, decoded it with `base64.b64decode`, converted the bytes to a NumPy array using `np.frombuffer`, and decoded it to a colour OpenCV image using `cv2.imdecode` with `cv2.IMREAD_COLOR`. It wrapped this in a try/except and raised an `HTTPException 400` if the decode failed.

`run_yolo_prediction` loaded the YOLO model from the weights path (loaded once at module level to avoid reloading on every request), called `model.predict` on the image array, and iterated over the boxes in `results[0].boxes`. For each detection it extracted the xyxy coordinates and converted them to x, y, width, height by computing `x = x1`, `y = y1`, `width = x2 minus x1`, `height = y2 minus y1`. Class labels were looked up from `results[0].names` using the integer class id, and probability was rounded to four decimal places.

The speed metrics were extracted from `results[0].speed` which is a dictionary with keys `preprocess`, `inference` and `postprocess`, each in milliseconds as a float. These were mapped to `speed_preprocess_ms`, `speed_inference_ms` and `speed_postprocess_ms` in the response.

`run_yolo_annotation` called `model.predict` similarly, then used `results[0].plot()` to get an annotated NumPy array with bounding boxes and labels drawn. This was encoded to JPEG bytes using `cv2.imencode` and then base64-encoded for the response.

`encode_image_to_base64` was a small helper that took a NumPy image array, encoded it with `cv2.imencode` and returned a base64 string, reused by the `annotate` helper.

Student Verification and Action

I tested all three endpoints with `demo-images/image0.jpeg`. `/api/predict` returned fire and smoke detections with valid bounding box coordinates and timing metrics. `/api/annotate` returned a base64 string that decoded to a correctly annotated JPEG.

4. Concurrency Handling for CPU-Bound YOLO Inference

Prompt

The assignment specification explicitly warns that running synchronous YOLO prediction inside an async FastAPI route blocks the event loop. Explain exactly what this means, how it causes latency spikes under

concurrent load, and show me the correct way to call CPU-heavy inference from an async endpoint. I need to be able to explain this clearly during the interview.

AI Response

The AI gave a detailed explanation. FastAPI uses an asyncio event loop that handles all incoming HTTP connections on a single thread. When a route is defined with `async def`, FastAPI expects that any waiting inside the function is done with `await` on a coroutine, which yields control back to the event loop so it can handle other requests while waiting.

YOLO inference is not a coroutine. It is a regular blocking Python function that runs matrix multiplications synchronously on the CPU and does not yield at any point. If you call it directly inside an `async def` route, the event loop is blocked for the entire duration of inference, typically 500 to 1500 milliseconds on a single vCPU. During that time the server cannot accept new connections, cannot read request data, and cannot send any responses. Every other concurrent user is effectively frozen.

The correct fix is to use `starlette.concurrency.run_in_threadpool`, which is a thin wrapper that submits the blocking call to FastAPI's default thread pool executor using `asyncio.get_event_loop().run_in_executor`. From the event loop's perspective the call becomes a coroutine that it can await without blocking, while the actual YOLO computation runs on a separate OS thread. The event loop remains free to handle other requests while inference is in progress.

The AI showed the import and usage. At the top: `from starlette.concurrency import run_in_threadpool`. Inside the route: `result = await run_in_threadpool(run_yolo_prediction, decoded_image)`. It clarified that this does not make inference faster on a single request, but it allows multiple requests to be in flight simultaneously, improving overall throughput under concurrent load.

It also noted an important caveat: because Python has a Global Interpreter Lock (GIL), only one thread can execute Python bytecode at a time. For pure Python code this limits the benefit, but NumPy and PyTorch operations release the GIL during their C-level computation, so genuine parallel inference is possible across multiple threads on a multi-core machine. On a single vCPU pod the benefit is mainly that the event loop stays responsive rather than achieving true parallel inference.

Student Verification and Action

I applied `run_in_threadpool` to both `/api/predict` and `/api/annotate` inference calls. I also prepared a clear spoken explanation of the event loop, blocking vs non-blocking calls, and the GIL for the interview.

5. Local Endpoint Test Scripts

Prompt

Create test scripts in a tests/ folder. Each script should load demo-images/image0.jpeg, encode it as base64, build the JSON payload with a sample UUID, and send a POST request to the endpoint. For predict: print the full JSON response. For annotate: validate that uuid and image fields exist in the response, decode the returned base64 image, and save it as tests/annotated_output.jpeg so I can visually verify the bounding boxes.

AI Response

The AI generated tests/test_predict.py and tests/test_annotate.py as standalone scripts using the requests library. Both scripts read image0.jpeg with open in binary mode, encoded it with base64.b64encode and decoded the result to a UTF-8 string to produce a plain text base64 string suitable for JSON.

test_predict.py built the payload as a Python dictionary with uuid set to a hardcoded sample UUID and image set to the encoded string, serialised it with json.dumps, and sent it as a POST to http://127.0.0.1:8000/api/predict. It printed the status code and then printed the JSON response using json.dumps with indent=2 for readability.

test_annotate.py sent the same payload to /api/annotate, checked that the response status code was 200, asserted that the response JSON contained both uuid and image keys, extracted the image value, decoded it from base64 back to bytes, and wrote those bytes to tests/annotated_output.jpeg using open in write-binary mode. It printed a confirmation message and the path of the saved file.

The AI suggested running the scripts from the project root as python tests/test_predict.py and python tests/test_annotate.py, and that the BASE_URL variable at the top could be changed from localhost to the public Oracle URL for remote testing.

Student Verification and Action

I ran both scripts locally against the running uvicorn server and then against the public NodePort URL http://169.224.230.182:30080. Both returned Status code: 200. The saved annotated_output.jpeg showed correct fire and smoke bounding boxes.

6. Dockerfile Engineering and Container Debugging

Prompt

Create a production-ready Dockerfile for the wildfire detection FastAPI app. I need python:3.11-slim as the base, CPU-only PyTorch installed with the correct index URL, dependency caching by copying requirements before source, a non-root user for security, YOLO_CONFIG_DIR set so the model does not try to write to a read-only path, port 8000 exposed, and uvicorn as the start command. Add a comment on every instruction so I can explain it in the interview. Also the container fails with ImportError: libxcb.so.1 when OpenCV tries to load, how do I fix this?

AI Response

The AI produced a fully commented Dockerfile. The FROM instruction used python:3.11-slim to start with a minimal Debian base that has only the Python runtime and no GUI libraries or build tools. WORKDIR was set to /app.

The first COPY was requirements-torch.txt followed by a RUN pip install --no-cache-dir --index-url https://download.pytorch.org/whl/cpu -r requirements-torch.txt. The separate requirements file contained only torch and torchvision so they would be installed from the CPU-only PyTorch index. Installing these first and separately from the rest of the requirements ensures Docker caches this expensive layer independently.

The second COPY was requirements.txt with the remaining dependencies (fastapi, uvicorn, ultralytics, opencv-python-headless, pydantic), followed by pip install --no-cache-dir -r requirements.txt.

For the libxcb.so.1 error, the AI diagnosed that opencv-python (the full version) links against X11 display libraries which are not present in a headless server container. The fix was twofold: switch requirements.txt to use opencv-python-headless instead of opencv-python, and add a RUN apt-get update and apt-get install -y --no-install-recommends libgl1 libglib2.0-0 to cover any remaining shared library dependencies. The headless variant removes all GUI-related bindings and is the correct choice for server deployments.

The COPY for source code came after all dependency layers so that changing app code does not invalidate the pip install cache. ENV YOLO_CONFIG_DIR=/tmp/yolo was set so Ultralytics writes its config to /tmp rather than ~/.config which may be read-only. RUN useradd -m appuser and USER appuser created and switched to a non-root user. EXPOSE 8000 documented the port. CMD started uvicorn on 0.0.0.0 so it was reachable from outside the container.

Representative Commands

```
docker build -t wildfire-api:local .  
docker run --rm -p 8000:8000 wildfire-api:local  
curl http://127.0.0.1:8000/health
```

Student Verification and Action

I rebuilt the image with docker build -t wildfire-api:local . after each change. After switching to opencv-python-headless the ImportError was resolved. I confirmed all three endpoints responded correctly inside the running container with curl.

7. Docker Hub Push and Registry Setup

Prompt

My image is built locally as wildfire-api:local. I need to push it to Docker Hub as suryaganesh2210/wildfire-api:latest so the Oracle Kubernetes worker nodes can pull it. Give me the exact commands and explain why the local image cannot be used directly by Kubernetes.

AI Response

The AI explained that Kubernetes worker nodes are separate virtual machines running in Oracle Cloud. They have no access to the Docker daemon on a developer's laptop. When a pod is scheduled, the kubelet on the worker node pulls the container image from a registry using the image name in the deployment manifest. If the image only exists locally on a laptop, the kubelet has no way to retrieve it and the pod will stay in ImagePullBackOff.

The solution is to push the image to a public registry such as Docker Hub. The commands were: docker login to authenticate, docker tag wildfire-api:local suryaganesh2210/wildfire-api:latest to create a tag that matches the Docker Hub repository path, docker push suryaganesh2210/wildfire-api:latest to upload the image layers, and docker images to verify the tag existed locally. The AI noted that imagePullPolicy: Always should be set in the Kubernetes deployment so the nodes always check for the latest version rather than using a cached older image.

Representative Commands

```
docker login
docker tag wildfire-api:local suryaganesh2210/wildfire-api:latest
docker push suryaganesh2210/wildfire-api:latest
```

Student Verification and Action

I logged into Docker Hub, ran the tag and push commands, and confirmed the image appeared in my Docker Hub repository. I then used suryaganesh2210/wildfire-api:latest as the image value in deployment.yaml.

8. Kubernetes Manifests and Cluster Deployment

Prompt

Create Kubernetes YAML files for the wildfire API deployment. I need a namespace called cloudeco, a deployment that pulls suryaganesh2210/wildfire-api:latest from Docker Hub, enforces exactly 1 vCPU and 2Gi memory requests and 4Gi memory limit per pod, uses app=wildfire-api as a label and selector, and includes /health readiness and liveness probes. Also create a service that exposes port 8000 on NodePort 30080. Add comments in the YAML so I can explain each field during the interview.

AI Response

The AI generated three files. namespace.yaml contained a single Namespace resource named cloudeco with a label app=cloudeco-system. It explained that using a namespace isolates all resources for this project and makes cleanup easier with kubectl delete namespace cloudeco.

deployment.yaml contained the Deployment resource in the cloudeco namespace. The spec had replicas set to 1 initially, a selector matching app=wildfire-api, and a pod template with the same label. The container spec had image: suryaganesh2210/wildfire-api:latest and imagePullPolicy: Always. The resources block had requests.cpu: "1" and requests.memory: "2Gi" and limits.cpu: "1" and limits.memory: "4Gi". The request and limit on CPU were both "1" which pins each pod to exactly one vCPU as required.

The readiness probe used httpGet on path /health port 8000 with initialDelaySeconds 30, periodSeconds 10 and failureThreshold 3. The 30-second initial delay allows the YOLO model weights to finish loading into memory before the probe fires. The liveness probe used the same path with initialDelaySeconds 60 to give the container extra time before Kubernetes considers restarting it. The AI explained that without readiness probes, the load balancer routes traffic to a pod that has started but has not yet loaded the model, causing the first few requests to fail.

service.yaml created a Service of type NodePort in the cloudeco namespace with selector app=wildfire-api, port 80 mapped to targetPort 8000, and nodePort 30080. This makes the API reachable from the internet at any worker node IP on port 30080.

Representative Commands

```
kubectl apply -f k8s/namespace.yaml
kubectl apply -f k8s/deployment.yaml
kubectl apply -f k8s/service.yaml
kubectl get pods -n cloudeco -o wide
```

Student Verification and Action

I validated each file with kubectl apply --dry-run=client -f k8s/ and then applied all three. After about 40 seconds the pod reached 1/1 Running status and /health responded correctly on the public URL.

9. Oracle Cloud Networking and CoreDNS Debugging

Prompt

After setting up the Kubernetes cluster on Oracle VMs the nodes show Ready but CoreDNS pods are 0/1 Running. kubectl logs on the CoreDNS pod shows dial tcp worker-ip:10250 no route to host. The public URL at http://169.224.230.182:30080 also times out even though the service looks correct inside the cluster. Diagnose both issues and give me the exact firewall rules and Kubernetes commands to fix them.

AI Response

The AI explained that the no route to host error on port 10250 is the kubelet API port. The master node needs to reach each worker's kubelet to check pod health, stream logs, and execute commands. Oracle Cloud's default security list blocks all traffic between VMs unless explicitly permitted. The fix required adding iptables rules on each VM to accept traffic from the pod network CIDR and the internal subnet, and also adding Oracle security list ingress rules.

For the internal firewall rules on each VM it recommended: `sudo iptables -I INPUT 1 -s 10.0.0.0/24 -j ACCEPT` for the internal VCN subnet, `sudo iptables -I INPUT 1 -s 10.244.0.0/16 -j ACCEPT` for the Flannel pod CIDR, `sudo iptables -I INPUT 1 -p tcp --dport 10250 -j ACCEPT` for the kubelet, `sudo iptables -I INPUT 1 -p udp --dport 8472 -j ACCEPT` for Flannel VXLAN, and `sudo iptables -I INPUT 1 -p tcp --dport 30080 -j ACCEPT` for the NodePort.

After applying these rules, `kubectl rollout restart deployment coredns -n kube-system` forces CoreDNS pods to reschedule. Once the kubelet ports are open the master can reach the workers and CoreDNS comes up to 1/1 Running.

For the public timeout, the AI explained that Oracle Cloud has a separate security list on the VCN subnet that controls external traffic independently of the VM's iptables. An ingress rule must be added in the Oracle Cloud console allowing TCP on port 30080 from source 0.0.0.0/0 on the public subnet associated with the worker node. Without this rule, traffic from the internet is dropped before it even reaches the VM.

Representative Commands

```
sudo iptables -I INPUT 1 -s 10.0.0.0/24 -j ACCEPT
sudo iptables -I INPUT 1 -s 10.244.0.0/16 -j ACCEPT
sudo iptables -I INPUT 1 -p tcp --dport 10250 -j ACCEPT
kubectl rollout restart deployment coredns -n kube-system
kubectl get pods -n kube-system
```

Student Verification and Action

I applied all the iptables rules on each VM, added the Oracle security list ingress rule for TCP 30080, and restarted CoreDNS. After about two minutes the CoreDNS pod reached 1/1 Running and `http://169.224.230.182:30080/health` returned 200 from my laptop.

10. Locust Load Generation Script

Prompt

Create a Locust script at `locust/locustfile.py`. It should load `demo-images/image0.jpeg` once at startup, convert it to base64, and simulate users sending POST requests to `/api/predict` and `/api/annotate`. Give `/api/predict` higher task weight since that is the main benchmark endpoint. Validate that the response contains the required fields and fail the task if any field is missing. Explain the `wait_time` choice and task weighting in comments.

AI Response

The AI generated a Locust `HttpUser` class named `WildfireUser`. At module level it opened `image0.jpeg` and encoded it once to avoid re-encoding on every simulated request, which would add unnecessary CPU overhead to the client and distort the server-side measurements.

wait_time was set to between(1, 3) seconds. The AI explained that this simulates a realistic user think time between requests. Setting it too low (e.g. between(0.1, 0.5)) produces an unrealistically aggressive arrival rate that can saturate the server before meaningful latency data is collected across multiple user levels.

The predict_endpoint task had a weight of 3 and the annotate_endpoint task had a weight of 1. This means approximately 75 percent of requests go to /api/predict. The AI explained that the assignment benchmarks are primarily around the predict endpoint and that annotate is slower because it also renders and encodes the annotated image, so weighting it lower keeps the benchmark representative of normal inference load.

Both tasks built the payload dict, called self.client.post with json=payload and name parameter for Locust grouping, and checked that response.status_code was 200. For predict, it then called response.json() and checked for uuid, count, detections, boxes, speed_preprocess_ms, speed_inference_ms and speed_postprocess_ms. For annotate it checked for uuid and image. If any check failed the task called response.failure with a descriptive message.

Representative Commands

```
python -m locust -f locust/locustfile.py --host  
http://169.224.230.182:30080
```

Student Verification and Action

I ran Locust with the web UI against the public Oracle URL and collected RPS, average latency, 95th percentile and failure counts for each pod replica setting.

11. Benchmark Design and the 6-Pod Decision

Prompt

The assignment asks for 1, 2, 4 and 8 pod tests. When I tried to scale to 8 pods some pods stayed in Pending with the message 0/2 nodes are available: 2 Insufficient memory. The two worker nodes each have 8GB RAM and the other pods on the cluster plus system pods consume some of it. The master node has a control-plane taint so pods do not schedule there by default. Should I lower the memory request to fit 8 pods, or report 6 pods? What is the correct academic approach?

AI Response

The AI explained that reducing the memory request to fit 8 pods would compromise the consistency of the benchmark. If earlier tests used requests.memory: 2Gi and the 8-pod test used a lower value, the pods are running under different resource constraints, which makes it impossible to fairly compare the performance results. The assignment rubric requires that resource limits be correctly enforced, and changing limits mid-experiment would undermine that.

The academically correct approach was to keep the resource configuration identical across all tests (cpu: "1", memory request 2Gi, memory limit 4Gi) and report the maximum fully schedulable configuration as 6 pods. The AI provided suggested wording for the report: "The cluster infrastructure consisted of two worker nodes each with 8GB RAM. System pods and DaemonSets consume approximately 1.5GB per node, leaving approximately 13GB schedulable memory across both workers. At a memory request of 2Gi per pod, the maximum schedulable replica count without modifying resource constraints was 6 pods. An 8-pod deployment produced Pending pods due to Insufficient memory on all available nodes. Results are therefore reported for 1, 2, 4 and 6 pods with consistent resource settings."

The AI also noted that documenting this clearly in the report demonstrates understanding of Kubernetes scheduling constraints and resource management, which aligns with the Distinction and HD criteria for section 4 of the rubric.

Representative Commands

```
kubectl scale deployment wildfire-api -n cloudeco --replicas=6
kubectl get pods -n cloudeco -o wide
kubectl describe pod -n cloudeco <pending-pod-name>
```

Student Verification and Action

I kept memory requests at 2Gi across all tests and benchmarked 1, 2, 4 and 6 pods. The 6-pod limitation was clearly documented in the report with the reason.

12. Benchmark Report Drafting and Analysis

Prompt

Help me write the 500-word benchmark report. My Locust results are: 1 pod stable at 5 users, 3044.97 ms average, 0.94 RPS. 2 pods stable at 10 users, 2604 ms average, 1.84 RPS. 4 pods stable at 20 users, 3546.12 ms average, 3.21 RPS. 6 pods stable at 20 users, 1668.26 ms average, 5.02 RPS. The report needs a results table, two plots (latency vs users and RPS vs users), and analysis using Little's Law and queuing theory. Keep it under 500 words.

AI Response

The AI structured the report in three parts. The results table had columns for pod count, max stable users, peak RPS and average response time. It noted that the 6-pod result at 20 users with a lower average latency (1668 ms) than the 4-pod result at 20 users (3546 ms) correctly shows that more pods reduce per-pod queue depth even at the same user count.

For the two plots it described: Plot 1 should map average response time on the y-axis against concurrent users on the x-axis, with one line per pod configuration. Plot 2 should map RPS on the y-axis against concurrent users. Both plots should show the saturation point where the lines flatten or spike.

For the analysis section, it applied Little's Law ($L = \lambda \times W$) to the 1-pod case: at 5 users each waiting an average of 3044 ms, the system contains approximately 0.94 times $3.044 = 2.86$ requests in service on average, confirming the pod is nearly fully occupied. When more users are added the queue grows because the service rate (0.94 RPS per pod) cannot increase beyond the CPU ceiling.

It identified the primary bottleneck as the synchronous YOLO inference consuming the entire vCPU for 500-1500 ms per request, making the effective service rate per pod roughly 0.9 requests per second. Horizontal pod scaling mitigates this because each additional pod adds an independent vCPU with its own service rate, so two pods can serve approximately 1.8 RPS and six pods approximately 5.4 RPS, consistent with the observed values.

The analysis also noted that the gains from 4 to 6 pods are sub-linear (3.21 to 5.02 RPS instead of the ideal 4.82 to 7.22 RPS), attributing this to scheduling overhead, inter-pod network latency, and the client-side Locust wait time limiting the arrival rate at the test boundaries.

Student Verification and Action

I used the AI draft as a starting point, verified all numbers against my actual Locust exports, trimmed the word count to under 500, and added the two plots generated from my data.

13. Terraform IaC Scripts for Oracle Cloud

Prompt

Create Terraform scripts to provision the Oracle Cloud infrastructure for the Kubernetes cluster. I need a VCN, public subnet, internet gateway, route table, security list allowing SSH and TCP 30080 inbound, and three compute instances (one master, two workers) using Oracle Linux 8. The scripts should use variables for region, compartment OCID and image OCID so they are not hardcoded. Explain each resource block.

AI Response

The AI generated four Terraform files. `variables.tf` declared input variables for `compartment_id`, `region`, `vm_image_ocid`, `vm_shape`, `ssh_public_key` and `cluster_cidr` with descriptions and type constraints. Having variables in a dedicated file means the actual values are supplied via a `terraform.tfvars` file that is not committed to source control, which is important for security.

`network.tf` created the VCN with `cidr_block` 10.0.0.0/24, the internet gateway attached to it, a route table with a default route pointing all traffic to the internet gateway, a security list with stateful ingress rules allowing TCP 22 from 0.0.0.0/0 for SSH and TCP 30080 from 0.0.0.0/0 for the NodePort, and a public subnet using the route table and security list. Each resource had a `display_name` prefixed with `cloudeco-`.

`compute.tf` created three `oci_core_instance` resources using `count = 1` for the master and `count = 2` for the workers, or alternatively three named resources. Each instance used `var.vm_shape` (e.g.

VM.Standard.E4.Flex), the public subnet, the Oracle Linux 8 image OCID, and a cloud-init script that set the SSH authorized key from var.ssh_public_key.

outputs.tf declared outputs for the public IP addresses of each instance so they could be used immediately after terraform apply without logging into the Oracle console.

The AI explained that running terraform init downloads the Oracle provider, terraform plan shows the execution plan without making changes, and terraform apply -auto-approve creates all resources in one command, which satisfies the assignment requirement for a single automated apply.

Student Verification and Action

I placed the scripts in a terraform/ directory, created a terraform.tfvars file with my compartment OCID, region and image OCID, ran terraform plan to verify the resource list, and then applied it. The VMs appeared in the Oracle console with public IPs matching the Terraform output.

14. Kubernetes Scaling Commands and Interview Demo Preparation

Prompt

I have a live interview where I need to demonstrate the cluster using kubectl, scale the deployment in real time, and run Locust to show the load balancer working. Give me a clear sequence of commands I should have ready in four terminal windows, and explain what each command shows and why it proves my implementation is correct.

AI Response

The AI designed a four-terminal layout. Terminal 1 was for kubectl observation commands to be run sequentially: kubectl get nodes -o wide to show all three VMs as Ready, kubectl get pods -n cloudeco -o wide to show pods and which worker node each is scheduled on, and kubectl get svc -n cloudeco to show the NodePort service and external port.

Terminal 2 was for scaling: kubectl scale deployment wildfire-api -n cloudeco --replicas=2 followed by watch kubectl get pods -n cloudeco to show new pods starting up and reaching Running status in real time. The AI noted that pointing at specific pods on different worker nodes proves the load balancer is distributing across machines.

Terminal 3 was for Locust: running python -m locust -f locust/locustfile.py --host http://169.224.230.182:30080 with the web UI open in a browser showing live RPS, latency and user count charts. Scaling from 1 to 2 to 4 replicas while Locust is running and observing the RPS increase live is strong evidence of working horizontal scaling.

Terminal 4 was for explaining the source code if asked: having main.py and utils.py open and being ready to scroll to the run_in_threadpool call, the resource limits in deployment.yaml, and the Locust task weights. The AI advised preparing a one-sentence explanation for each: what run_in_threadpool does and why it matters, why CPU limit is "1", and why predict has a higher weight than annotate.

It also suggested preparing to explain the Terraform apply output and show the oracle VMs in the console, demonstrating IaC provisioning even if the VMs were already running by the interview time.

Representative Commands

```
kubectl get nodes -o wide
kubectl get pods -n cloudeco -o wide
kubectl scale deployment wildfire-api -n cloudeco --replicas=2
watch kubectl get pods -n cloudeco
python -m locust -f locust/locustfile.py --host
http://169.224.230.182:30080
```

Student Verification and Action

I practiced the full demo sequence twice before the interview, timing each section. I kept all four terminals arranged on screen and tested that all commands worked from the current shell environment with the correct kubeconfig loaded.

15. Request Payload Validation and Image Decoding

Prompt

My API accepts a base64 image inside a JSON body. I want to make sure invalid inputs are rejected gracefully rather than causing a crash deep inside the YOLO model. How should I structure the Pydantic model and the image decoding function to handle bad data properly? Also, some clients send base64 strings with a data URL prefix like data:image/jpeg;base64, — should I strip that?

AI Response

The AI explained that Pydantic is the right place to enforce that both uuid and image fields are present and are strings. If either field is missing, FastAPI automatically returns HTTP 422 Unprocessable Entity with a clear JSON error message before any application code runs. This covers the basic case of a malformed request body.

For the data URL prefix, the AI confirmed this is common when browsers or frontend tools encode images. The fix is a single line at the start of the decode function: if the string contains a comma, split on the comma and take the second part. This strips everything up to and including data:image/jpeg;base64, without breaking plain base64 strings that have no prefix.

The more critical validation is inside the decode function itself. After running `base64.b64decode` and `np.frombuffer`, the result of `cv2.imdecode` should be checked for `None`. `imdecode` returns `None` when the bytes are not a valid image format, for example if the client sent a base64-encoded text file or a corrupted upload. The AI showed wrapping the decode in a try/except block and raising `HTTPException` with `status_code=400` and a detail message like "Invalid image data: could not decode base64 content" when `imdecode` returns `None`. This gives the client a clear 400 Bad Request instead of a 500 Internal Server Error from a null pointer inside YOLO.

It also noted that Pydantic v2 (used in newer FastAPI versions) supports field validators with the `@field_validator` decorator if more granular checks are needed, for example checking that the base64 string is not empty before attempting to decode.

Student Verification and Action

I added the data URL prefix strip and the `imdecode None` check to `app/utils.py`. I tested by sending a plain text string as the image field and confirmed the API returned 400 with the correct error message rather than crashing.

16. YOLO Bounding Box Coordinate Conversion

Prompt

The Ultralytics YOLO result gives me bounding boxes in `xyxy` format which is `x1, y1, x2, y2` corner coordinates. The assignment requires `x, y, width, height` format. Show me the exact conversion and explain which corner `x` and `y` represent in the output format.

AI Response

The AI clarified that in the assignment response format, `x` and `y` represent the top-left corner of the bounding box, not the centre. This is different from the YOLO training format (YOLO .txt format) which uses normalised centre `x`, centre `y`, width, height. The Ultralytics `result.bboxes.xyxy` already gives absolute pixel corner coordinates, so no normalisation is needed.

The conversion from `xyxy` to `xywh` with top-left origin is: `x = x1`, `y = y1`, `width = x2 minus x1`, `height = y2 minus y1`. In Python using the Ultralytics box tensor: `x1, y1, x2, y2 = box.xyxy[0].tolist()`, then `x = round(x1, 2)`, `y = round(y1, 2)`, `width = round(x2 - x1, 2)`, `height = round(y2 - y1, 2)`.

The probability value comes from `box.conf[0].item()` which returns a float between 0 and 1. The AI recommended rounding to four decimal places with `round(prob, 4)` for clean JSON output. The class label is retrieved with `result.names[int(box.cls[0].item())]` where `result.names` is the dictionary mapping integer class IDs to string labels such as "fire" and "smoke".

It also noted that when count is 0 (no detections), the boxes field should be an empty list, not null. The code should initialise boxes as an empty list and only append to it inside the detection loop, so the response is always a valid JSON array regardless of whether any objects were detected.

Student Verification and Action

I implemented the conversion in app/utils.py and tested with image0.jpeg. The returned boxes had correct x, y, width, height values that visually aligned with the bounding boxes drawn in the annotated image.

17. Dockerfile Layer Caching and CPU-Only PyTorch

Prompt

Why does the order of COPY and RUN instructions in a Dockerfile matter for build speed? And how do I install PyTorch CPU-only to avoid pulling a huge CUDA build that I don't need since the pods only have 1 vCPU and no GPU?

AI Response

The AI explained Docker layer caching. Docker builds an image as a stack of layers, one per instruction. Each layer is cached based on its content and the content of all layers before it. When you rebuild an image, Docker checks each layer from top to bottom. The first layer whose content has changed, and all layers below it, are rebuilt. Layers above the changed layer are reused from cache.

This means instruction order has a large practical impact on rebuild time. If requirements.txt is copied and dependencies are installed early, those layers are cached and reused on every rebuild as long as requirements.txt does not change. If source code is copied before dependencies, any code change invalidates the dependency install layer and forces a full pip install on every rebuild, which can take several minutes for PyTorch.

The correct order is: copy requirements.txt first, run pip install, then copy the application source code. This way changing a Python file only invalidates the final COPY layer and rebuilds in seconds. The expensive pip install layer is only rebuilt when requirements.txt itself changes.

For CPU-only PyTorch, the AI showed the correct pip install command: `pip install --no-cache-dir torch torchvision --index-url https://download.pytorch.org/whl/cpu`. Without the `--index-url` flag, pip defaults to PyPI which distributes the CUDA version of PyTorch. The CUDA build is approximately 2 to 3 GB whereas the CPU build is approximately 200 MB. In a Kubernetes deployment with multiple worker nodes pulling the image, this difference multiplies across every pull and significantly affects startup time.

Student Verification and Action

I restructured the Dockerfile to copy requirements first, verified the layer order with docker history wildfire-api:local, and confirmed the CPU-only torch was installed by checking the image size with docker images. The final image was approximately 2.1 GB compared to over 5 GB with the CUDA PyTorch.

18. Kubernetes Readiness and Liveness Probe Design

Prompt

Explain the difference between a readiness probe and a liveness probe in Kubernetes and why both matter for a YOLO inference service specifically. What values should I use for initialDelaySeconds given that the YOLO model takes time to load at startup?

AI Response

The AI explained the distinction clearly. A liveness probe tells Kubernetes whether the container is still alive. If the liveness probe fails enough times, Kubernetes kills the container and restarts it. A readiness probe tells Kubernetes whether the container is ready to receive traffic. If the readiness probe fails, the pod is removed from the service endpoints and no traffic is routed to it, but the container is not killed.

For a YOLO inference service this distinction is critical at startup. When a new pod starts, the Ultralytics YOLO model must be loaded from disk into memory. On a 1 vCPU container this can take 15 to 40 seconds depending on the model size. During this loading time, the container process is running (so a liveness probe would pass) but any request to /api/predict would either fail or return a 500 error because the model object is not yet initialised.

Without a readiness probe, the Kubernetes service would immediately route traffic to the new pod the moment its container starts, causing the first batch of requests to fail. With a readiness probe set to initialDelaySeconds 30 and pointing at /health (which should return 200 only after the model finishes loading), traffic is only sent to the pod after it has confirmed it is ready.

The AI recommended setting initialDelaySeconds to 30 for the readiness probe and 60 for the liveness probe. The longer liveness delay prevents Kubernetes from killing the pod during initial startup if model loading takes longer than expected. periodSeconds of 10 and failureThreshold of 3 gives the pod 30 seconds to recover from a transient failure before it is restarted.

It also suggested that /health should be implemented to actually verify the model is loaded, for example by checking that the global model variable is not None, rather than just returning a static healthy string. This makes the probe a meaningful health check rather than just a port check.

Student Verification and Action

I implemented both probes in deployment.yaml with the recommended delay values and updated /health in app/main.py to check that the YOLO model object was loaded before returning 200. I confirmed by

watching kubectl describe pod during scaling events that pods only appeared in the service endpoints after the readiness probe passed.

19. Terraform Oracle Cloud Network and Compute Resources

Prompt

Help me understand what Terraform resources I need for the Oracle Cloud (OCI) infrastructure. I need a VCN, public subnet, internet gateway, route table, security list with the right ports open, and three compute instances. The region should be ap-melbourne-1. What is the OCI Terraform provider structure and what are the most important fields for each resource?

AI Response

The AI outlined the OCI Terraform provider setup. The provider block requires `tenancy_ocid`, `user_ocid`, `fingerprint`, `private_key_path` and `region`. For a student deployment using environment variables (`OCI_CLI_AUTH=api_key`), these can be sourced from the OCI CLI config file using the `config_file_profile` attribute instead.

The networking resources follow a specific creation order because of dependencies. The `oci_core_vcn` is created first with `cidr_block` 10.0.0.0/24 and `dns_label`. The `oci_core_internet_gateway` is attached to the VCN. The `oci_core_route_table` contains a route rule with destination 0.0.0.0/0 pointing to the internet gateway OCID. The `oci_core_security_list` defines ingress rules for TCP 22 (SSH), TCP 6443 (Kubernetes API), TCP 10250 (kubelet), TCP 8472 UDP (Flannel VXLAN), and TCP 30080 (NodePort), plus an egress rule for all traffic. The `oci_core_subnet` ties the route table and security list together.

For compute instances, `oci_core_instance` resources specify `availability_domain`, `compartment_id`, `shape` (VM.Standard.E5.Flex), `source_details` referencing the Ubuntu 22.04 image OCID, `create_vnic_details` with the subnet OCID and `assign_public_ip` true, and `shape_config` with `ocpus` 4 and `memory_in_gbs` 8. The `metadata` block passes the SSH public key as `ssh_authorized_keys`.

The AI recommended using a `locals` block to define the image OCID and availability domain so they appear once and are referenced throughout, and a `variables.tf` file for `compartment_id`, `ssh_public_key` and any values that change between deployments. `outputs.tf` should output the public IP of each instance using `oci_core_instance.master.public_ip` and similar references.

Representative Commands

```
cd Iac && terraform init
terraform plan -out=tfplan
terraform apply tfplan
terraform output
```

Student Verification and Action

I placed all Terraform files in the Iac/ directory, created terraform.tfvars with my compartment OCID and SSH public key, ran terraform init, terraform plan, and terraform apply. The three VMs appeared in the Oracle console with the correct shape and the public IPs matched the Terraform outputs.

20. Understanding Cluster Memory Constraints and the 6-Pod Decision

Prompt

I tried to scale to 8 pods but some pods stayed in Pending with the message 0/2 nodes are available: 2 Insufficient memory. My two worker VMs each have 8GB RAM and my pods request 2Gi each. Mathematically 8 pods times 2Gi is 16Gi which equals 16GB, so why can't it schedule? And what is the cleanest way to explain this in my report and interview without it looking like I failed?

AI Response

The AI explained the gap between raw VM memory and Kubernetes schedulable memory. Each worker node runs Ubuntu 22.04 as the operating system, which uses approximately 500MB to 800MB. On top of that, Kubernetes runs the kubelet process, the container runtime (containerd), kube-proxy, Flannel for pod networking, CoreDNS, and potentially a metrics server or log collector. These system components together consume 1GB to 1.5GB per node in a typical self-managed cluster.

Kubernetes also reserves a portion of node memory using the --kube-reserved and --system-reserved kubelet flags, and applies an eviction threshold (default 100Mi) below which it will not schedule new pods. With a typical small cluster setup, approximately 2GB of each 8GB worker node is unavailable for pod scheduling. This leaves approximately 6GB schedulable per worker, or 12GB across both workers. At 2Gi request per pod, the maximum schedulable count is 6 pods, which exactly matches the deployment result.

For the report, the AI provided a suggested paragraph: "The assignment specification required testing at 8 pods. The OCI deployment used two worker VMs each with 8GB memory. Kubernetes system components, the container runtime, and the OS together consume approximately 2GB per node, leaving approximately 12GB of schedulable memory across both workers. At a pod memory request of 2Gi, the maximum schedulable replica count was 6 pods. Scaling to 8 replicas produced Pending pods due to Insufficient memory on all available nodes. Results are therefore reported for 1, 2, 4, and 6 pods with consistent resource settings (1 vCPU, 2Gi request, 4Gi limit per pod)."

For the interview, the AI advised being direct and technical: explain what schedulable memory is, why it differs from raw RAM, and use kubectl describe node to show the Allocatable memory field live, which proves the constraint is real and understood.

Representative Commands

```
kubectl describe node <worker-node-name>
kubectl get nodes -o jsonpath="{.items[*].status.allocatable.memory}"
```

```
kubectl scale deployment wildfire-api -n cloudeco --replicas=6
kubectl get pods -n cloudeco -o wide
```

Student Verification and Action

I used this explanation in both the benchmark report and prepared it as a spoken answer for the interview. I ran `kubectl describe node` on each worker to confirm the Allocatable memory value and included the output in my submission evidence.

21. Locust Benchmark Evidence Collection and CSV Export

Prompt

How should I collect and save my Locust benchmark results so I have proper evidence for the report submission? I want to record RPS, average response time, median, P95 and failure percentage for each pod configuration. Is there a way to export from Locust without manually copying numbers from the screen?

AI Response

The AI explained that Locust has a built-in CSV export that writes three files: a stats file, a stats history file, and a failures file. Running Locust in headless mode with the `--csv` flag produces these automatically. The stats file contains per-endpoint and aggregate rows with request count, failure count, average, min, max, median, P95 and P99 response times, and RPS.

The recommended approach for benchmark collection was to run one test per pod configuration, wait for the metrics to stabilise at the target user count (at least 60 seconds of stable data), then either download the CSV from the Locust web UI download button or stop the headless run which finalises the CSV files. Screenshot the RPS and response time charts from the Locust web UI before stopping, as these show the time-series behaviour that the static CSV rows cannot capture.

For the report table, the AI suggested using the Aggregated row from the stats CSV for each pod count because it combines both the predict and annotate endpoints into a single average, which is appropriate for overall system characterisation. For the plots, the stats_history CSV contains timestamped rows and can be used to plot the latency trajectory as users increase.

It also advised recording the exact number of users and spawn rate used for each test so the test conditions are reproducible and can be described precisely in the report.

Representative Commands

```
python -m locust -f locust/locustfile.py --host
http://169.224.230.182:30080 --users 10 --spawn-rate 1 --run-time 3m --
csv=results_1pod --headless
```

```
python -m locust -f locust/locustfile.py --host
http://169.224.230.182:30080 --users 20 --spawn-rate 1 --run-time 3m --
csv=results_2pod --headless
```

Student Verification and Action

I ran each pod configuration test for at least two minutes at the target user count and exported the CSV from the Locust web UI. I also took screenshots of the charts at peak load for each configuration. The CSV data was used to populate the results table in the benchmark report.

22. Security Practices in the Container and Cloud Deployment

Prompt

What security practices should I apply to the Docker container and Oracle Cloud deployment for this assignment? I know the pod should not run as root. What else should I do and how do I explain the security decisions during the interview?

AI Response

The AI listed the practical security steps relevant to the assignment scope. In the Dockerfile, creating a dedicated non-root user with `useradd -m appuser` and switching to it with `USER appuser` prevents any process inside the container from having root privileges on the host node if a container escape occurs. This is a standard hardening step for production containers and is specifically mentioned in the HD criteria of the rubric.

Setting `YOLO_CONFIG_DIR` to `/tmp/Ultralytics` (or `/tmp/yolo`) as an environment variable in the Dockerfile prevents Ultralytics from trying to write to the home directory of a potentially read-only user, and also means the config path is predictable and contained to the tmp filesystem rather than mixed with application files.

For Oracle Cloud, the security list on the public subnet should be as restrictive as the assignment allows. The minimum required open ports are TCP 22 for SSH (ideally restricted to your IP rather than 0.0.0.0/0 for a real deployment), TCP 30080 for the NodePort service, and internal cluster ports within the VCN CIDR. The AI noted that for an assignment demo environment, opening 30080 from any source is acceptable but should be noted as something that would be changed in production.

The AI also mentioned that `imagePullPolicy: Always` in the deployment manifest ensures worker nodes always pull the latest image from Docker Hub rather than using a potentially outdated cached version. While this is slightly slower for pod startup, it is safer for correctness during development.

For the interview, the AI suggested framing the security explanation as a three-level answer: container level (non-root user, no unnecessary packages), Kubernetes level (resource limits prevent a pod from consuming all node resources), and network level (security list restricts external access to only the required ports).

Student Verification and Action

All security practices were already implemented during the Dockerfile and Terraform phases. I prepared the three-level interview explanation and practiced describing each decision in plain terms without relying on jargon alone.

23. README and Submission Package Preparation

Prompt

Help me write the README_A1.md file. It needs to cover: the public URL, how to call each endpoint, how to build and run Docker locally, how to apply the Kubernetes manifests, how to run the scaling commands for each pod configuration, how to run Locust, and how to run the Terraform IaC. Also clearly document the 6-pod limitation.

AI Response

The AI structured the README with eight sections. The first section listed the public deployment URL as `http://169.224.230.182:30080` and confirmed that `/health` returns status healthy.

The endpoints section listed `GET /health`, `POST /api/predict` and `POST /api/annotate` with the JSON payload schema for the two POST endpoints. It included a curl example for `/api/predict` showing the Content-Type header and a minimal JSON body with uuid and a placeholder for the base64 image string.

The Docker section covered `docker build -t wildfire-api:local .` followed by `docker run --rm -p 8000:8000 wildfire-api:local` and a note that the container serves on `0.0.0.0:8000` inside and maps to the host port.

The Kubernetes section listed `kubectl apply -f k8s/namespace.yaml`, `kubectl apply -f k8s/deployment.yaml` and `kubectl apply -f k8s/service.yaml` in order, followed by `kubectl get pods -n cloudeco -o wide` to verify the pod is running. The scaling section gave `kubectl scale` commands for each replica count: 1, 2, 4 and 6.

The Locust section gave the run command with the public host URL and noted that the web UI opens at `http://localhost:8089` where users and spawn rate can be set interactively.

The Terraform section covered `cd Iac`, `terraform init`, `terraform plan` and `terraform apply`, with a note to populate `terraform.tfvars` with the compartment OCID and SSH public key before running.

For the 6-pod limitation the README included a clear paragraph explaining that 8 pods could not be scheduled because two worker nodes at 8GB each leave approximately 12GB schedulable after OS and Kubernetes overhead, and 8 pods at 2Gi request each require 16Gi, exceeding available capacity. The maximum stable configuration was 6 pods and all benchmark results use this as the upper bound.

Student Verification and Action

I used the AI draft as a starting point and edited it to match the actual commands and file paths in my repository. I verified every command listed in the README actually worked from a clean terminal before finalising the file.

24. Interview Preparation – Technical Questions and Explanations

Prompt

I have a 20-minute live technical interview where I need to explain my own code and demonstrate the cluster. What technical questions am I likely to be asked and how should I answer them clearly? I want to cover: why `run_in_threadpool` is used, how the load balancer distributes traffic, what CPU limit 1 means in the YAML, why readiness probes matter, and how Little's Law explains the latency spikes.

AI Response

The AI went through each topic. For `run_in_threadpool`: the expected question is "why did you use `run_in_threadpool` instead of just calling the YOLO model directly?" The answer is that FastAPI runs on an async event loop which is single-threaded. YOLO inference is a blocking CPU operation. If called directly inside an async route, the event loop is blocked for the entire inference duration and cannot process any other requests. `run_in_threadpool` submits the blocking call to a thread pool so the event loop stays free to accept and handle other requests concurrently.

For load balancer traffic distribution: the Kubernetes Service of type NodePort (or LoadBalancer) uses kube-proxy on each node to implement IP tables rules that distribute incoming connections across the ready pod endpoints using round-robin by default. The interviewer may ask to demonstrate this by checking that requests go to different pods. This can be shown by logging the pod name in each response and watching multiple requests in the Locust output.

For CPU limit 1: the YAML value "1" means one CPU core, equivalent to 1000 millicores or 1000m. The request and limit being equal pins the pod to exactly one vCPU, which is required by the assignment for valid benchmarking because it ensures each pod has the same processing power and the RPS results are comparable across replica counts.

For readiness probes: the expected question is "what happens without a readiness probe during scaling?" The answer is that Kubernetes routes traffic to new pods as soon as the container starts, before the YOLO model has loaded. Requests fail during that window. With a readiness probe, the pod only enters the service endpoints after `/health` returns 200, which is only configured to happen after model loading completes.

For Little's Law: L equals λ times W , where L is the average number of requests in the system, λ is the arrival rate, and W is the average time each request spends in the system. When arrival rate exceeds service rate, the queue grows. More pods increase service rate because each adds an independent vCPU. The

AI suggested drawing this as a simple diagram on paper during the interview if the interviewer asks for a visual explanation.

Student Verification and Action

I practiced each answer out loud several times. I also prepared a one-page printed summary of the key commands so I could run them quickly during the demo without searching through terminal history under pressure.
